



B-TREE VE B+TREE AĞAÇLARININ GÜNÜMÜZDE KULLANIM ALANLARI

Veritabanları, Dosya Sistemleri ve Büyük Veri Sistemlerinde
İndeksleme

[FEYZANUR UYSAL]
[VERİ ORGANİZASYONU]
[18.05.2026]



B-TREE VE B+TREE YAPILARI

B-TREE

- Her düğümde anahtar + veri bulunur
- Yapraklar arasında bağlantı yok

B+TREE

- Veri sadece yapraklarda tutulur
- Yapraklar birbirine bağlı listedir
- Sıralı aramalar çok hızlıdır

ORTAK ÖZELLİK

- Kendini dengeler, disk uyumlu, $O(\log n)$ arama



GÜNLÜK HAYAT BENZETMESİ: KÜTÜPHANE SİSTEMİ

KÜTÜPHANE = VERİTABANI

Kitaplar = Veriler

Katalog sistemi = B+Tree

B-TREE GİBİ OLAN (Eski sistem):

- Her rafta kitap ve detayı birlikte
- Tarama yaparken çok raf gezmek gerekir

B+TREE GİBİ OLAN (Modern sistem):

- Ana koridorlar: Sadece konu başlıkları ve raf numarası (iç düğümler)
- Her rafın sonunda bir sonraki rafa bağlantı (yaprak bağlantısı)
- Tüm kitapları sırayla dolaşmak çok hızlıdır

Faydası: "Romanlar" diye aratınca direkt koridoru bulursunuz.



VERİTABANLARINDA B+TREE KULLANIMI

MYSQL (InnoDB)

- B+Tree kullanır
- PRIMARY KEY = clustered index (veri yaprakta sıralı)
- Range sorguları (BETWEEN, >, <) çok hızlıdır

POSTGRESQL

- Varsayılan indeks tipi B-Tree (B+Tree benzeri)
- Yaprak blokları arasında bağlantı tutar

MONGODB (WiredTiger)

- B+Tree kullanır
- _id üzerinde varsayılan indeks B+Tree'dir



DOSYA SİSTEMLERİNDE B-TREE KULLANIMI

NTFS, EXT4, XFS gibi dosya sistemleri

dizin indeksleme (directory indexing) için B-Tree kullanır.

EXT4 ÖRNEĞİ:

- HTree adı verilen bir B-Tree türü kullanılır
- Büyük dizinlerde (örnek: 1 milyon dosya) hızlı arama sağlar

NASIL ÇALIŞIR?

- Bir dosya arandığında tüm dizini taramak yerine
B-Tree üzerinde logaritmik arama yapılır



ARAMA MOTORLARI VE BÜYÜK VERİ SİSTEMLERİ

ELASTICSEARCH / LUCENE

- Ters indeks (inverted index) oluşturur
- Sıralama ve metadata için B-Tree benzeri yapılar kullanır

GOOGLE BIGTABLE / HBASE

- Ana veri yapısı LSM Tree'dir (yazma odaklı)
- Ancak metadata (tablet konumları) B-Tree ile tutulur

CASSANDRA

- Partition key yönlendirmesinde B+Tree mantığı kullanılır



NEDEN B-TREE VE B+TREE TERCİH EDİLİYOR?

1. DİSK UYUMLU

Düğüm boyutu disk sayfası (4KB) ile eşleşir → az I/O

2. KARARLI YÜKSEKLİK

Milyonlarca veride bile 3-4 seviyede arama yapılır

3. RANGE SORGULARI İÇİN İDEAL

Özellikle B+Tree'de yaprak bağlantıları sayesinde

4. OTOMATİK DENGELİ (SELF-BALANCING)

Veri eklenip silinse de performans sabit kalır

5. LOCK CONTENTION AZ

B+Tree'de sadece yaprak düğümlere yazılır



SORU 1: BU VERİ YAPILARI NEDEN ÖNEMLİDİR?

B-Tree ve B+Tree olmasaydı, bir veritabanında arama yapmak için tüm satırları tek tek taramak zorunda kalırdık. Bu yapılar sayesinde milyonlarca kayıt içinde bile saniyeler içinde sonuç bulunabiliyor. Benzer şekilde bilgisayarımızda bir dosya ararken de bu ağaçlar arka planda çalışarak işlemleri hızlandırıyor. Bu yüzden modern bilgisayar biliminin temel yapı taşlarından biridir.



SORU 2: BÜYÜK VERİ ÇAĞINDA NEDEN İHTİYAÇ DUYULMAKTADIR?

Günümüzde sosyal medya, bankacılık, e-ticaret gibi alanlarda saniyede milyonlarca işlem yapılıyor. Bu verilerin tamamını RAM'de tutmak imkansız çünkü disk boyutları çok büyük. B+Tree sayesinde veri diske yazılırken bile hızlı erişim sağlanıyor. Ayrıca büyük veride "son 1 aydaki siparişler" gibi aralık sorguları çok önemlidir; B+Tree'nin yaprak bağlantıları bu sorguları çok hızlı yapar.



SORU 3: GELECEKTE DE KULLANILMAYA DEVAM EDECEK Mİ?

Bence evet, ama biraz değişerek. Şu anda yazma odaklı sistemlerde LSM Tree (Log Structured Merge Tree) daha popüler hale geliyor (Cassandra, RocksDB gibi). Ancak okuma ağırlıklı ve range sorgusu çok olan sistemlerde (MySQL, PostgreSQL, dosya sistemleri) B+Tree uzun süre daha kullanılmaya devam edecek. Gelecekte yapay zeka ile hangi indeksin daha iyi olacağını tahmin eden sistemler çıkabilir, ama B-Tree'nin temel mantığı ölmeyecek.



TEŞEKKÜRLER

Dinlediğiniz için teşekkürler.

Sorularınızı beklerim.